基于业务流程的待集成系统规划文档

一、 规划依据与系统边界

本规划基于四大核心业务流程（学习ISA、工业芯片验证、设计ISA、开发机器语言程序）中的“软件协助需求”提炼而成。

系统边界声明：本系统定位于**软件级ISA仿真与基准验证**。在工业芯片验证环节，系统**不负责**集成芯片逻辑仿真器（EDA工具/DUT），而是作为**黄金参考模型（Golden ISS）**，提供标准执行轨迹与比对服务。

二、 待集成的信息系统（软件工具链与开源组件）

根据业务需求，系统需集成以下成熟的信息系统组件，以快速实现核心功能：

1. 汇编与代码编辑系统（面向机器语言程序员、学习者）

- **需求来源**：“程序编写、翻译和链接”、“选定后自动语法检查”。
- **待集成系统**：
  - **开源交叉汇编器与链接器（如 GNU Binutils 或 LLVM 工具链）**：用于将用户编写的汇编代码翻译为 `.bin` / `.hex` 机器码，并支持链接脚本（Linker Script）进行物理地址分配。
  - **代码编辑器组件（如 `Monaco Editor`）**：嵌入Web前端，提供语法高亮、行号、断点标记，并可在前端集成轻量级的语法检查器（Linter）。

2. 程序加载与解析系统（面向所有用户）

- **需求来源**：“运行环境模拟（编写Loader）”、“清零BSS段”。
- **待集成系统**：
  - **ELF/BIN 解析库（如 `libelf` 或 `elfio`）**：用于解析用户上传的可执行文件，提取代码段（.text）、数据段（.data）和符号表，并正确加载至模拟器的虚拟内存中。

3. 核心仿真与追踪系统（面向学习者、ISA工程师）

- **需求来源**：“指令执行和执行现场模拟”、“指令执行时信号追踪”。
- **待集成系统**：
  - **反汇编引擎（如 `Capstone`）**：实时将十六进制机器码翻译为汇编助记符，支撑前端指令行高亮。
  - **Trace 追踪与导出系统**：在仿真核心中加入日志探针，记录每一条指令执行时的 PC、寄存器堆、内存变化（生成结构化 JSON/文本），用于前端展示和对外提供标准比对轨迹。

4. 调试与可视化协议系统（面向机器语言程序员、工业工程师）

- **需求来源**：“利用调试器进行单步执行”、“执行现场可视化”。
- **待集成系统**：
  - **标准调试协议（GDB RSP）**：在C++核心中实现远程串行协议。这样前端可以通过 WebSocket 转发调试指令（断点、单步、读取寄存器），复用标准的调试后端逻辑。
  - **内存/寄存器可视化组件（Hex Viewer 等）**：在Web端实现十六进制与ASCII双栏显示，支持地址跳转和变化字节高亮。

5. 自动化比对与验证系统（面向工业芯片工程师、ISA工程师）

- **需求来源**：“Difftest（黄金参考模型比对）”、“语法一致性监督”。
- **待集成系统**：
  - **Diff 比对引擎**：提供日志比对工具，允许用户上传其RTL仿真产生的Trace，与本系统生成的Golden Trace进行逐行比对。
  - **自动化脚本框架（如 Python + Pytest）**：用于批量运行标准测试集（如 `riscv-tests`）。
  - *(开发期限定)* **黄金参考模型（如 Spike）**：**仅在研发和CI/CD阶段集成**用于验证自研模拟器的准确性。**系统发布版本中不包含。**

6. 文档与知识库系统（面向ISA学习者）

- **需求来源**：“内置指令手册，可查阅，有例程及解释”。
- **待集成系统**：
  - **静态站点生成器或文档框架（如 VuePress / Docusaurus）**：用于组织ISA指令集手册、例程代码及解析文档，与模拟器前端界面互相链接。

三、 待集成的物理系统（宿主环境与硬件映射）

作为课程大作业，物理系统的集成主要围绕“宿主计算资源”与“软硬件接口映射”展开：

1. 宿主操作系统与文件系统

- **需求来源**：“加载与运行环境构建”。
- **集成方式**：直接利用物理宿主机（Linux/Windows/Mac）的文件读写能力，管理用户上传的固件、汇编文件及导出的比对日志。

2. Web 浏览器物理沙盒（Wasm 运行环境）

- **需求来源**：降低网络延迟，提高单步执行响应速度。
- **集成方式**：将 C++ 核心通过 `Emscripten` 编译为 WebAssembly (Wasm)。用户的物理浏览器直接执行模拟指令，通过前端 JS 直接读取内存快照更新 UI。

3. 虚拟外设接口（MMIO）

- **需求来源**：“运行环境模拟”。
- **集成方式**：在虚拟内存地址空间中划分一段区域（Memory-Mapped I/O）。当机器码程序向该特定物理地址写入数据时，系统捕获该操作，并将其模拟为终端的字符输出（UART）或LED状态变化。

四、 系统架构集成总结图景

- **前端交互层**：集成 Monaco Editor（编写）、Hex Viewer（内存查看）、WebSocket（通信）。
- **核心仿真层**：集成 Capstone（反汇编）、GDB RSP（调试协议）、ELF Loader（加载）。
- **数据比对层**：集成 Trace 生成器、Python 自动化比对脚本。
- **底层环境**：利用物理宿主文件系统，探索 Wasm 浏览器沙盒加速。